

ML pentru Olimpiada

De ce

Toate modelele sunt gresite, dar unele sunt utile.

– George Box

Inca ezit putin sa ma apuc de el deoarece:

Nu sunt foarte convins ca numele *AI* descrie ceea ce facem prin aceste metode. Mi se pare, de multe ori, o umbra palida a cognitiei: luam lucruri extrem de complexe si le trecem printr-un model surogat si miop, optimizat pentru un proxy care nu surprinde neaparat ceea ce ne intereseaza cu adevarat. Iar termenul este folosit foarte des cu o conotatie mult mai pretentioasa decat continutul real.

Mai important, cred ca o olimpiada de ML risca sa puna prea mult accent pe **cum facem ceva** si prea putin pe **de ce facem acel lucru**.

La informatica, o metrica este de obicei destul de cinstita: o solutie corecta este corecta. In ML, lucrurile sunt mai ample. Poti ajunge sa optimizezi o metrica care este doar un proxy pentru un obiectiv mult mai complex, iar raspunsul la o problema poate deveni pur si simplu “facem o retea mai mare”. In industrie se ajunge uneori sa construim sisteme care optimizeaza foarte bine lucruri care poate nu ar fi trebuit optimizate in primul rand.

De asta vreau ca acest curs sa fie mai mult decat o colectie de algoritmi si trucuri pentru olimpiada.

Magia apare atunci cand intelegi ce poti sa rezolvi, cum poti sa rezolvi si, mai ales, de ce merita sa rezolvi problema respectiva.

Creierul nu e facut pentru statistica

Traim in cea mai sigura perioada din istoria omenirii, si totusi ne e frica de aproape orice. Motivul nu e ca lumea reala e periculoasa, e ca noi, ca specie, suntem prost echipati sa intelegem probabilitatile. Creierul judeca riscul dupa cat de vie e o poveste, nu dupa cat de des se intampla ceva de fapt. Ne e frica de avion, unde riscul de deces e undeva la 1 la cateva milioane de zboruri, dar urcam linistiti la volan, unde riscul e cu mult mai mare, doar ca nu are aceeasi incarcatura emotionala.

Acelasi mecanism il vezi la loterie: oamenii aleg numere “norocoase” sau evita 13, desi fiecare combinatie are exact aceeasi sansa. Il vezi la ruleta, cand dupa opt rosii la rand toata lumea e convinsa ca urmeaza negru, desi bila nu tine minte nimic din rotirile anterioare (asta se cheama *falacia jucatorului*). Si il vezi la poker: un jucator bun castiga pe termen lung nu pentru ca ghiceste cartile, ci pentru ca ia, mana dupa mana, decizii

corecte statistic, chiar daca pierde uneori o mana anume. Diferenta dintre un jucator bun si unul prost nu e norocul de moment, e intelegerea probabilitatilor pe termen lung, si o sa exersam exact chestia asta la clasa, cu carti pe masa, nu doar cu formule pe tabla.

De ce conteaza asta pentru un curs de ML? Pentru ca jobul vostru, ca oameni care lucreaza cu date, e exact opusul acestui reflex: sa scoateti decizia din mana intuitiei si sa o puneti pe o baza masurabila. Dar ca sa faceti asta bine, trebuie sa fiti foarte atenti de unde vin datele alea.

Garbage in, garbage out

Poate ati observat ca studiile de nutritie se contrazic constant: intr-un an oule sunt periculoase, in urmatorul sunt super-aliment. Motivul, de multe ori, nu e ca stiinta se schimba, e ca datele sunt proaste de la inceput. Multe studii se bazeaza pe chestionare in care oamenii isi amintesc ce au mancat saptamana trecuta, memorie nesigura, si raporteaza ce cred ca ar trebui sa fi mancat, nu ce au mancat cu adevarat.

Nu conteaza cat de sofisticat e modelul vostru: o retea neuronală cu sute de milioane de parametri, antrenata pe date subiective, tot zgomot invata, nu realitate. Algoritmul e o oglinda, reflecta exact ce ii dai, buguri si minciuni incluse. De-aia o sa insistam, in cursurile urmatoare, pe cum se colecteaza si se curata datele, nu doar pe cum se antreneaza un model.

Un exemplu istoric bun pentru genul asta de eroare, dar dintr-un unghi diferit, e *Clever Hans*, un cal din Germania de la inceputul secolului XX care parea sa stie sa faca operatii matematice, batand din copita de cate ori era nevoie. Nu stia matematica, desigur: citea, fara sa vrea nimeni asta, semnale corporale involuntare de la antrenorul lui, care se relaxa vizibil cand calul ajungea la numarul corect. Asta se cheama, azi, *data leakage* sau *shortcut learning*: modelul, sau calul, invata sa exploateze un semnal care e corelat cu raspunsul corect, dar care n-are nicio legatura reala cu problema, si care de multe ori nici n-o sa mai fie disponibil cand rulezi modelul in productie. Ideea merita retinuta devreme: cand un model merge suspect de bine, prima intrebare nu e “ce genial e”, e “ce a gasit, de fapt, sa trisheze”.

Corelatie nu inseamna cauzalitate

Japonezii mananca putina carne si traiesc foarte mult. Spaniolii mananca de doua ori mai multa carne, de patru ori mai multa grasime animala, si totusi sunt pe locul doi in lume la speranta de viata, la o distanta infima de Japonia. Daca ai antrena un model simplu care leaga direct consumul de carne de mortalitate, ai rata complet explicatia.

Ce lipseste din poza e tot ce nu ai masurat: sistemul de sanatate, cat mers pe jos face fiecare zi, cat de relaxat e contextul social in care mananci. Astea se numesc *variabile de confuzie*, factori ascunsi care influenteaza si cauza aparenta si efectul, dar pe care nu i-ai inclus in date. Un model nu stie ce ai uitat sa masori, el vede doar ce ii dai, si gaseste corelatii, nu explicatii. Partea grea a lucrului cu date nu e sa rulezi un algoritm, e sa te gandesti ce lipseste din tabelul tau inainte sa tragi o concluzie.

Metrica gresita, decizie gresita

Un exemplu la scara mare: dupa accidentul de la Fukushima, unde numarul de decese directe cauzate de radiatii a fost aproape zero, mai multe tari au inchis centrale nucleare functionale, de frica unui accident extrem de rar, si au trecut, partial, pe carbune. Poluarea din arderea carbunelui insa e o cauza constanta si masurabila de mortalitate, an de an, la o scara mult mai mare decat orice accident nuclear din istorie. Optimizand impotriva riscului mai spectaculos, dar mai rar, ai putea agrava riscul mai banal, dar mai frecvent si mai mare.

Asta e exact problema unei functii de cost (*loss function*) prost alese intr-un model de ML: daca optimizezi pentru metrica gresita, modelul tau o sa “rezolve” cu succes o problema care nu era cea reala. O sa vorbim mult despre metrici mai incolo in curs, accuracy, precision, recall si restul, dar tineti minte de pe acum: alegerea metricii nu e un detaliu tehnic, e decizia care conteaza cel mai mult.

Exista si o lege scurta care rezuma toata problema asta: cand o masura devine tinta, ea inceteaza sa mai fie o masura buna, *legea lui Goodhart*, formulata ulterior si *Campbell*: cu cat un indicator cantitativ social este folosit mai mult pentru luarea deciziilor, cu atat va fi mai supus presiunilor de coruptie si va distorsiona procesul pe care intentioneaza sa il monitorizeze. O vezi peste tot: note care devin scopul scolii, in loc de invatare; KPI-uri de companie care se optimizeaza fara sa mai reflecte performanta reala; retele sociale care optimizeaza timpul petrecut pe aplicatie, nu binele utilizatorului. Fiecare model de ML pe care il construiti o sa optimizeze exact ce ii cereti, nici un pic mai mult si nici un pic mai intelept. Responsabilitatea sa cereti lucrul corect ramane a voastra.

Cum o sa invatam

O sa incepem cu ceva simplu si contraintuitiv, ca sa vedeti cat de prost va poate insela intuitia: problema Monty Hall, cu cele trei usi, unde raspunsul corect nu e cel pe care il da aproape toata lumea din prima. E un exercitiu de cinci minute care merita mai mult decat o ora de teorie.

N-o sa fie doar teorie si cod pe ecran. O sa incerc sa va invat meseria asta prin exercitii practice, si multe dintre ele o sa fie imprumutate din jocuri: o sa jucam barbut, o sa jucam poker, si o sa calculam impreuna sansele reale din spatele fiecarei decizii. Un joc de noroc e, de fapt, un laborator perfect de probabilitati: regulile sunt simple, miza e clara, si poti sa masori exact cat de bine gandesti fata de cat de bine crezi ca gandesti.

O sa vorbim si despre lucruri din viata de zi cu zi, nu doar despre exemple abstracte de manual. Vreau sa incepeti sa observati cat de des va loviti de fapt de rationament probabilistic, chiar si atunci cand nu il numiti asa: cand decideti daca luati umbrela, cand cantariti un risc, cand cititi o stire cu un procent aruncat fara context.

O sa atingem si zone mai fine de optimizare matematica, dar mereu cu intrebarea in minte: cat de des ne lovim de fapt de genul asta de problema? Nu vreau sa invatati o formula doar ca sa o uitati dupa concurs, vreau sa recunoasteti situatia in care chiar merita folosita.

Un obiectiv mai ambitios: la finalul cursului, sper sa puteti purta un debate corect cu cineva, pe baza de date si probabilitati, nu pe baza de cine vorbeste mai tare sau cine are

povestea mai convingatoare. Asta e, pana la urma, toata ideea: sa inlocuisti argumentul emotional cu unul care rezista la verificare.

O nota de filozofie

Undeva la baza tuturor acestora sta o intrebare mai veche decat calculatoarele: cum ar trebui sa gandim in conditii de incertitudine? Filozofii au numit-o *problema inductiei* cu multe secole inainte sa existe Machine Learning: din faptul ca soarele a rasarit in fiecare zi din viata ta, nu poti fi sigur 100% ca rasare si maine, doar foarte, foarte increzator. Toata statistica moderna, si tot ce construim noi in cursul asta, e un raspuns practic la intrebarea aia: cum cuantificam increderea, in loc sa o lasam la nivel de senzatie.

Nu e o intrebare doar tehnica. E, in fond, o intrebare despre onestitate intelectuala: esti dispus sa iti schimbi parerea cand apar date noi, sau te agati de prima poveste care ti s-a parut convingatoare? Un model bun de ML face asta automat, isi actualizeaza increderea pe masura ce vede date noi. Un om rational ar trebui sa faca la fel.

Multe dintre debaterile de azi despre inteligenta artificiala si transumanism nu sunt, de fapt, chiar noi. Intrebarea daca o masina poate sa “gandeasca” cu adevarat, daca poate avea constiinta, daca deciziile ei sunt cu adevarat ale ei sau doar rezultatul mecanic al unor cauze anterioare, sunt variante reimpachetate ale unor debateri vechi de mii de ani: ce e sufletul, exista liber arbitru sau totul e determinism, ce ne face de fapt oameni. Filozofii dezbateau determinismul cu mult inainte sa existe retele neuronale, teologii dezbateau natura sufletului cu mult inainte sa existe calculatoare. Tehnologia noua ne da un vocabular nou pentru intrebari vechi, nu neaparat raspunsuri noi la ele.

N-o sa rezolvam aici, in doua paragrafe, ce n-au rezolvat filozofii in doua mii de ani. Dar data viitoare cand cineva va vinde o poveste dramatica despre o inteligenta artificiala care ne va inlocui sau despre masini constiente, intrebati-va cate din argumentele alea sunt de fapt argumente teologice vechi, rescrise cu cuvinte noi.

Asta nu inseamna sa subestimati unealta: ML rezolva azi probleme pe care acum douazeci de ani le credeam imposibile, si ne ajuta sa evoluam, nu doar sa fim mai eficienti. Nu vreau sa iesiti din curs nici naiv-entuziasmati, nici sperati de ziua de maine, ci suficient de lamuriti incat sa folositi unealta cu incredere, nu cu superstitie.

Ce incerc sa insirui

Un parcurs de la zero pana la nivel de olimpiada, care va evolua pe parcurs. E facut din ce descriu ei pe ONIA, dar nu imi place ordinea in care se face asta neaparat.

- Python, NumPy, Pandas
- Prelucrarea si vizualizarea datelor
- Machine Learning fundamentals
- Regresie si clasificare
- Decision Trees, Random Forest, Gradient Boosting, SVM, KNN
- Clustering si PCA

- Feature engineering
- Train/validation/test si cross-validation
- Overfitting, underfitting si regularizare
- Metrici: Accuracy, Precision, Recall, F1, ROC-AUC, MSE, MAE, RMSE, R^2
- Hyperparameter tuning
- Neural Networks si PyTorch
- Computer Vision
- NLP
- Probleme si exercitii de tip olimpiada

Structura exacta nu este batuta in cuie. Cursul se va construi pe masura ce vedem ce este util, ce este dificil si ce merita aprofundat.

Referinte

- [Programa oficiala a Olimpiadei Nationale de Inteligenta Artificiala](#)

Contents

De ce	1
I Curs 0: Introducere in Python	9
1 Introducere in Python	10
1.1 De ce Python, nu C++	10
1.2 Cum ar trebui sa scriem Python	10
1.3 Sintaxa pe repede inainte	11
1.4 Exercitii	12
1.4.1 Suma cifrelor unui numar	12
1.4.2 Suma elementelor unui vector	12
1.4.3 Maximul dintr-un vector	12
1.4.4 Numararea elementelor pare	12
II Curs 1: NumPy si Pandas	13
2 NumPy si Pandas	14
2.1 De ce NumPy	14
2.2 Ce face v.mean() de fapt	14
2.3 Complexitate: aceeasi, dar...	15
2.4 Cum masuram timpul de executie	15
2.5 De ce Pandas	16
2.6 Exercitii	16
2.6.1 Statistici fara bucla	16
2.6.2 Suma si maximul, cu NumPy	17
2.6.3 Sortare si elemente unice	17
III Apendice	18
3 Coduri: Curs 0	19
3.1 Suma cifrelor unui numar	19
3.2 Suma elementelor unui vector	19
3.3 Maximul dintr-un vector	19
3.4 Numararea elementelor pare	20

4	Coduri: Curs 1	21
4.1	Statistici fara bucla	21
4.2	Suma si maximul, cu NumPy	21
4.3	Sortare si elemente unice	21

Part I

Curs 0: Introdúcere in Python

Chapter 1

Introducere in Python

O sa presupun ca stii deja cum se parcurg vectori si matrici, ai facut asta de multe ori la C++. Fara presupunerea asta ne-am duce prea mult in spate cu recapitularea, si nu la asta ne intereseaza timpul. Ce facem aici e sa traducem ce stii deja in sintaxa lui Python, cat mai repede, ca sa ajungem la partea interesanta.

1.1 De ce Python, nu C++

Intrebarea e corecta. Ai invatat C++ pentru olimpiada pentru ca e rapid, apropiat de masina, te lasa sa controlezi fiecare octet. De ce am schimba asta acum?

Raspunsul scurt: nu il schimbam, il ascundem. Aproape tot ce faci in NumPy sau PyTorch ruleaza, pe dedesubt, in cod C sau C++ compilat, ai vazut deja asta in [capitolul urmator](#) cand comparam `v.mean()` cu bucla scrisa de mana. Python nu inlocuieste C++, il apeleaza.

Ce castigi renuntand la C++ ca limbaj de zi cu zi:

- scrii mai putin cod pentru aceeasi idee, deci iterezi mai repede;
- ai acces la un ecosistem urias de librarii facute exact pentru ML (NumPy, Pandas, scikit-learn, PyTorch);
- gresesti mai rar la lucruri marunte (memorie, tipuri), pentru ca Python le gestioneaza singur.

Ce pierzi: viteza bruta pe cod scris de mana. Dar in ML aproape niciodata nu scrii tu bucla critica, o scrie altcineva o data, in C, si tu o apelezi de mii de ori. De-aia insist sa intelegi ce e in spate, nu ca sa scrii tu C prin Python, ci ca sa stii cand o functie e doar sirop peste ceva simplu si cand chiar face treaba grea pe care n-ai vrea sa o repeti singur.

1.2 Cum ar trebui sa scriem Python

Cateva reguli care difera fata de C++, si pe care merita sa le ai in cap de la inceput:

- Blocurile de cod sunt definite prin indentare, nu prin acolade. Daca indentarea e gresita, codul e gresit.
- Nu declari tipul unei variabile. `x = 5` e de ajuns, Python afla singur ca `x` e intreg.

- Nu pui punct si virgula la finalul liniei.
- Numele de variabile si functii se scriu `snake_case`, nu `camelCase`.
- Daca te trezesti scriind o bucla explicita peste o lista mare de date, intreaba-te intai daca nu exista deja o functie care face asta, o sa revenim serios la ideea asta in capitolul urmator.

1.3 Sintaxa pe repede inainte

Trecem prin echivalentele directe ale lucrurilor pe care le stii deja din C++. Nu explic ce e o bucla sau ce e o functie, doar cum arata in Python.

```
# variabile, fara tip declarat
n = 5
nume = "Alex"
gasit = True

# if / else
if n > 0:
    print("pozitiv")
elif n == 0:
    print("zero")
else:
    print("negativ")

# bucla, echivalentul lui for (int i = 0; i < n; i++)
for i in range(n):
    print(i)

# bucla, echivalentul lui for (int x : v)
v = [1, 2, 3]
for x in v:
    print(x)

# functie
def patrat(x):
    return x * x
```

`list`-ul e echivalentul lui `vector` din C++, doar ca poate tine orice tip de date amestecat si isi schimba singur dimensiunea:

```
v = []
v.append(10)
v.append(20)
print(len(v))    # 2
print(v[0])      # 10
```

Operatorul `%` (rest) si operatorul `//` (cat la impartirea intreaga) fac exact ce fac `%` si `/` intre intregi in C++, doar ca in Python `/` intre intregi da mereu rezultat zecimal, de-aia exista `//` separat pentru impartirea intreaga.

O sa vezi in capitolul urmator ca, pentru date numerice, NumPy inlocuieste `list`-ul cu ceva mai apropiat, ca structura de memorie, de `vector`-ul din C++. Dar pana acolo,

`list`-ul e suficient.

1.4 Exercitii

Facem cativa algoritmi de baza, din cei pe care ii stii deja din C++, dar scrisi acum in Python, de mana, cu bucla explicita. Daca ai mai citit prin preajma, stii ca nu iti trebuie neaparat forma asta, o sa vedem in [capitolul urmator](#) ca NumPy face acelasi lucru intr-o singura linie. Dar e important sa trecem prin ele de mana, macar o data, inainte sa le facem cu o librerie: altfel folosesti `numpy.sum()` fara sa stii ce calculeaza de fapt.

1.4.1 Suma cifrelor unui numar

[notebook](#) • [surse](#)

Extragem cifrele unui numar una cate una, cu impartire si rest la 10, si le adunam. Genul de exercitiu pe care l-ai facut deja de zeci de ori in C++, doar sintaxa difera.

1.4.2 Suma elementelor unui vector

[notebook](#) • [surse](#)

Parcurgem lista element cu element si acumulam suma. O sa refacem exact exercitiul asta in capitolul urmator, cu `numpy.sum()`, ca sa vezi diferenta de cod, si de viteza.

1.4.3 Maximul dintr-un vector

[notebook](#) • [surse](#)

Aceeasi idee ca la maximul dintr-un vector in C++: retinem cel mai mare element vazut pana acum, pe masura ce parcurgem lista.

1.4.4 Numararea elementelor pare

[notebook](#) • [surse](#)

Parcurgem lista si numaram cate elemente indeplinesc o conditie, aici, sunt pare. E un tipar care revine des: parcurgere plus conditie plus contor.

Part II

Curs 1: NumPy si Pandas

Chapter 2

NumPy si Pandas

Pentru inceput, un lucru destul de simplu, dar care conteaza mai mult decat pare la prima vedere: cum tine Python datele in memorie, si de ce, la un moment dat, un simplu `list` nu mai e de ajuns.

2.1 De ce NumPy

Cand scrii `v = [1, 2, 3, 4]` in Python, nu obtii un bloc de numere asezate frumos unul langa altul in memorie, cum ai crede. Obtii o lista de referinte, fiecare aratand catre cate un obiect Python separat, cu tot overhead-ul lui. E flexibil, dar e risipa cand ai milioane de numere de acelasi fel.

`numpy.ndarray` rezolva exact asta: tine date brute, de un singur tip, intr-un bloc continuu de memorie. Dar motivul pentru care merita sa inveti bine NumPy nu e viteza in sine, e ca te obliga sa gandesti altfel.

In loc sa ii spui calculatorului, pas cu pas, cum sa parcurga fiecare element, ii spui ce vrei sa obtii, pe tot vectorul deodata. Asta se cheama *vectorizare*, regula asta o sa te salveze de multe ori: daca te trezesti ca scrii un `for` peste un array NumPy, aproape sigur exista deja o operatie care face exact ce vrei tu, mai rapid si mai clar.

```
import numpy as np

v = np.array([1, 2, 3, 4])
v * 2      # array([2, 4, 6, 8]), fara bucla
v.mean()   # 2.5
v[v > 2]    # array([3, 4]), indexare booleana
```

Nu trebuie sa memorezi functiile astea! Nici la olimpiada nu se asteapta nimeni sa le stie pe de rost, ai voie cu documentatia acolo. O sa vezi singur, din exercitii, care le folosesti des si care nu.

2.2 Ce face `v.mean()` de fapt

Aici e partea care conteaza cu adevarat pentru noi. `mean()` nu e magie, e o suma impartita la un numar, atat:


```
def medie(v):
    s = 0
    for x in v:
        s += x
    return s / len(v)
```

Asta e tot ce face NumPy pe dinauntru. Doar ca bucla ruleaza intr-un cod C, peste memorie continua, nu in Python peste obiecte separate, de-aia e de zeci de ori mai rapid, nu pentru ca ar calcula altceva. Vezi [exercitiul](#) de mai jos pentru masuratoarea reala.

Ideea se repeta la aproape orice functie din NumPy sau Pandas: `sum`, `std`, `groupby().mean()`, toate sunt formule simple, scrise o singura data, corect si rapid, ca sa nu le rescrii tu de fiecare data. Merita sa stii sa le scrii si de mana, macar o data. Pe asta ne concentram in cursul asta, nu doar pe ce functie exista si cum se apeleaza.

2.3 Complexitate: aceeasi, dar...

Amandoua variantele sunt $\mathcal{O}(n)$: fiecare element e vizitat o singura data, fie in bucla scrisa de mana, fie pe dinauntru lui `v.mean()`. Din punctul de vedere al complexitatii, sunt identice.

Bucla de mana, $\mathcal{O}(n)$:

```
s = 0
for x in v:
    s += x
medie = s / len(v)
```

NumPy, $\mathcal{O}(n)$:

```
medie = v.mean()
```

Atunci de unde diferenta de viteza pe care o masuram mai jos? Complexitatea iti spune cum creste timpul de executie o data cu n , nu cat dureaza efectiv un pas. Fiecare pas din bucla Python inseamna: interpretorul citeste o instructiune, verifica tipul lui `x`, face un apel de functie pentru `+=`, etc. Fiecare pas din `v.mean()` inseamna o singura adunare in cod masina, pe date deja asezate in memorie. Constanta ascunsa in spatele lui $\mathcal{O}(n)$ e mult mai mica la NumPy, chiar daca forma curbei e aceeasi.

De-aia complexitatea nu e de ajuns: iti spune daca un algoritm se degradeaza bine cand creste n , dar nu iti spune care varianta e mai rapida la un n fix. Pentru asta, masori direct.

2.4 Cum masuram timpul de executie

Am zis mai sus ca varianta vectorizata e "de zeci de ori mai rapida", dar nu ti-am aratat cum masor asta de fapt. Nu e nimic ascuns: retii timpul inainte, retii timpul dupa, faci diferenta.

```
import time

start = time.perf_counter()
medie(v)
timp = time.perf_counter() - start
```

```
print(timp)    # cat a durat, in secunde
```

`time.perf_counter()` iti da un ceas cat mai precis, facut special pentru masuratori de genul asta (spre deosebire de `time.time()`, care e legat de ceasul sistemului de operare si poate sari inainte sau inapoi, de exemplu la sincronizare de retea). Diferenta dintre cele doua momente e timpul de rulare, atat.

Ca sa compari doua variante, faci exact acelasi lucru de doua ori si raportezi una la alta:

```
start = time.perf_counter()
medie(v)
timp_bucla = time.perf_counter() - start

start = time.perf_counter()
v.mean()
timp_numpy = time.perf_counter() - start

print(f"bucla: {timp_bucla:.4f}s")
print(f"numpy: {timp_numpy:.4f}s")
print(f"de {timp_bucla / timp_numpy:.0f}x mai rapid")
```

Daca lucrezi intr-un notebook Jupyter, ca cele din exercitii, exista si o comanda mai scurta, `%timeit`, care repeta automat masuratoarea de mai multe ori si iti da o medie mai stabila decat o singura masuratoare de mana. E convenabila, dar functioneaza doar in Jupyter, nu si intr-un script .py obisnuit, de-aia merita sa stii si varianta cu `time.perf_counter()`: e cea pe care o poti pune oriunde.

2.5 De ce Pandas

Un `DataFrame` e un tabel: coloane tipate, fiecare coloana fiind pe dedesubt tot un `ndarray`, si randuri indexate. E unealta pentru date tabelare, citire CSV, filtrare, grupare, agregare, pe care o sa o folosim la orice set de date cu care lucram, inainte sa ajungem macar la vreun model.

```
import pandas as pd

df = pd.read_csv("date.csv")
df[df["scor"] > 50]           # filtrare
df.groupby("categorie")["scor"].mean() # agregare
```

2.6 Exercitii

2.6.1 Statistici fara bucla

[notebook](#) • [surse](#)

Calculam medie, deviatie standard si normalizare pe un vector, intai cu bucla `for` ca sa vedem exact ce calculeaza, apoi vectorizat cu NumPy, si comparam timpul de executie intre variante.

2.6.2 Suma si maximul, cu NumPy

[notebook](#) • [surse](#)

Acelasi [suma](#) si [maxim](#) din Curs 0, de data asta cu `v.sum()` si `v.max()`. Compara codul cu bucla de mana pe care ai scris-o acolo, calculul e identic, doar scris o singura data, in libreria.

2.6.3 Sortare si elemente unice

[notebook](#) • [surse](#)

`np.sort()` sorteaza un vector, `np.unique()` scoate duplicatele (si il sorteaza pe deasupra), `np.argmax()` iti da pozitia maximului, nu valoarea lui. Trei functii pe care o sa le folosesti des la prelucrarea datelor.

Part III

Apendice

Chapter 3

Coduri: Curs 0

3.1 Suma cifrelor unui numar

```
def suma_cifre(n):  
    s = 0  
    while n > 0:  
        s += n % 10  
        n //= 10  
    return s  
  
suma_cifre(12345) # 15
```

3.2 Suma elementelor unui vector

```
def suma(v):  
    s = 0  
    for x in v:  
        s += x  
    return s  
  
suma([1, 2, 3, 4, 5]) # 15
```

3.3 Maximul dintr-un vector

```
def maxim(v):  
    m = v[0]  
    for x in v[1:]:  
        if x > m:  
            m = x  
    return m  
  
maxim([3, 7, 2, 9, 4]) # 9
```

3.4 Numararea elementelor pare

```
def numara_pare(v):  
    cnt = 0  
    for x in v:  
        if x % 2 == 0:  
            cnt += 1  
    return cnt  
  
numara_pare([1, 2, 3, 4, 5, 6]) # 3
```

Chapter 4

Coduri: Curs 1

4.1 Statistici fara bucla

```
import numpy as np

v = np.random.rand(1_000_000)

# varianta cu bucla - asta calculeaza de fapt v.mean()
def medie_bucla(v):
    s = 0.0
    for x in v:
        s += x
    return s / len(v)

# varianta vectorizata, acelasi rezultat, de zeci de ori mai rapid
v.mean()

# normalizare (z-score): aducem datele la medie 0, deviatie 1
z = (v - v.mean()) / v.std()
```

4.2 Suma si maximul, cu NumPy

```
import numpy as np

v = np.array([3, 7, 2, 9, 4])

v.sum()    # 25, acelasi rezultat ca suma() scrisa de mana
v.max()    # 9, acelasi rezultat ca maxim() scrisa de mana
```

4.3 Sortare si elemente unice

```
import numpy as np

v = np.array([5, 2, 8, 2, 5, 1, 9])
```

```
np.sort(v)      # array([1, 2, 2, 5, 5, 8, 9])
np.unique(v)    # array([1, 2, 5, 8, 9]), sortat si fara duplicate
np.argmax(v)    # 6, pozitia lui 9 in v, nu valoarea lui
```
